

Tab này chưa kết nối PMW3901, chỉ test khả năng generate random point của EPS32 lên màn hình

1. 2 đoạn code ban này làm gì(Cả arduino và processing)?
2. Vì sao cần phải chạy code trên 2 IDE?
3. Sơ đồ khối và sơ đồ luồng hoạt động. Đưa code mermaid để dán vào draw.io
4. Trả lời ngắn gọn. Giai đoạn tiếp theo làm gì?

1. Hai đoạn code ban này làm nhiệm vụ gì?

Code Arduino (chạy trên vi điều khiển ESP32-S3):

- **Nhiệm vụ:** Đóng vai trò là trạm thu thập và phát dữ liệu (Transmitter).
- **Chi tiết:** Nó tạo ra một mảng gồm 1225 giá trị ngẫu nhiên (từ 0-255) để giả lập độ sáng của 1225 điểm ảnh. Sau đó, nó gửi ký tự * làm cờ đánh dấu "bắt đầu khung hình", tiếp theo là bản toàn bộ 1225 byte này qua cáp USB lên máy tính.

Code Processing (chạy trên Máy tính):

- **Nhiệm vụ:** Đóng vai trò là trạm thu nhận và render đồ họa (Receiver & Renderer).
- **Chi tiết:** Nó liên tục nghe ngóng cổng COM18. Khi bắt gặp cờ *, nó biết một khung hình mới đang tới. Nó sẽ hứng chính xác 1225 byte tiếp theo, lưu vào bộ đệm, sau đó quy đổi từng byte thành mức độ xám (0 = đen, 255 = trắng) và vẽ ra một ma trận ô vuông 35x35 trên màn hình.

2. Vì sao phải chạy code trên 2 IDE?

Đây là mô hình **Client - Server** hoặc **Hardware - Interface** kinh điển trong hệ thống nhúng, phân chia rạch ròi nhiệm vụ vì đặc thù của từng phần cứng:

- **ESP32 (Arduino IDE):** Vi điều khiển có các chuẩn giao tiếp phần cứng thấp (SPI, I2C, GPIO) để đọc tín hiệu điện từ cảm biến PMW3901 theo thời gian thực (điều mà máy tính PC không cắm dây trực tiếp vào được). Tuy nhiên, nó không có màn hình và khả năng xử lý đồ họa rất yếu.
- **PC (Processing IDE):** Máy tính có sức mạnh đồ họa vượt trội, dư sức vẽ hàng ngàn pixel mượt mà, nhưng lại thiếu các cổng giao tiếp phần cứng (chỉ có USB).
- **Kết luận:** ESP32 làm nhiệm vụ "kéo" dữ liệu phần cứng và đẩy qua USB. Processing nhận dữ liệu từ USB và "vẽ" ra cho con người xem.

3. Sơ đồ khối và Sơ đồ luồng (Mermaid)

3.1 Sơ đồ khối



flowchart LR

%% Định nghĩa các khối chính

subgraph Hardware ["Phần cứng bên ngoài"]

PMW[Cảm biến quang học PMW3901]

end

```

subgraph MCU ["Vi điều khiển: ESP32-S3 Supermini"]
    SPI_Controller[Module SPI<br>Master]
    Mem[SRAM<br>Frame Buffer 1225 Bytes]
    USB_CDC[Giao tiếp<br>USB CDC / UART TX]
end

```

```

subgraph PC ["Máy tính (Host)"]
    COM_Port[Cổng COM ảo<br>UART RX]
    App[Processing App<br>Java State Machine]
    Screen[Màn hình máy tính<br>Canvas 525x525]
end

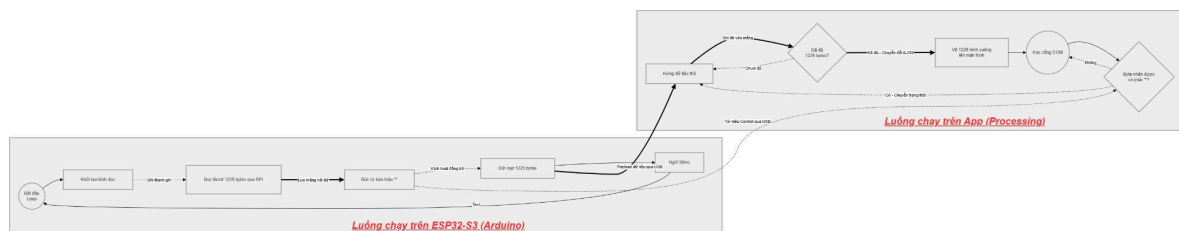
```

```

%% Liên kết giữa các khối
PMW <==>|Bus SPI vật lý| SPI_Controller
SPI_Controller -->|Lưu trữ tạm thời| Mem
Mem -->|Chuyển dữ liệu| USB_CDC
USB_CDC <==>|Cáp USB Type-C| COM_Port
COM_Port -->|Luồng byte| App
App -->|Render đồ họa| Screen

```

3.2 Sơ đồ luồng



flowchart LR

```

%% Subgraph ESP32
subgraph Trạm phát ["Luồng chạy trên ESP32-S3 (Arduino)"]
    direction LR
    E_Start((Bắt đầu<br>Loop)) --> E_Req[Khởi tạo lệnh đọc]
    E_Req -->|Ghi thành ghi| E_Read[Đọc Burst 1225 bytes qua SPI]
    E_Read ==>|Lưu trữ nội bộ| E_Flag[Gửi cờ báo hiệu '*']
    E_Flag -->|Kích hoạt đồng bộ| E_Data[Gửi loạt 1225 bytes]
    E_Data --> E_Delay[Nghỉ 50ms]
    E_Delay --> E_Start
end

%% Subgraph Máy tính
subgraph Trạm thu ["Luồng chạy trên App (Processing)"]
    direction LR
    P_Start((Đọc cổng COM)) --> P_CheckFlag[Byte nhận được<br>có phải '*'?]

    P_CheckFlag -->|Không| P_Start
    P_CheckFlag -->|Có - Chuyển trạng thái| P_HưngData[Hưng dữ liệu thô]

    P_HưngData ==>|Ghi đè vào mảng| P_CheckĐủ{Đã đủ<br>1225 bytes?}

    P_CheckĐủ -->|Chưa đủ| P_HưngData

```

```
P_CheckĐủ ==>|Đã đủ - Chuyển đổi 0-255| P_Render[Vẽ 1225 hình vuông<br>lên  
màn hình]
```

```
P_Render --> P_Start  
end
```

```
%% Liên kết giữa 2 luồng (Truyền thông)  
E_Flag -.->|Tín hiệu Control qua USB| P_CheckFlag  
E_Data ==>|Payload dữ liệu qua USB| P_HứngData
```

4. Giai đoạn tiếp theo làm gì? (Ngắn gọn)

1. **Đấu nối phần cứng:** Nối các chân SPI (MOSI, MISO, SCK, CS) của PMW3901 vào ESP32-S3.
2. **Lập trình giao tiếp SPI:** Xóa hàm `random()` trong code Arduino. Viết các hàm giao tiếp trực tiếp vào thanh ghi (bare-metal) hoặc dùng thư viện để đọc dữ liệu Frame Grab thực tế từ cảm biến PMW3901 đổ vào mảng `frameBuffer`. Cấu trúc truyền dữ liệu (`*` và 1225 bytes) giữ nguyên.